

Relatório final do laboratório 01 de medição e experimentação de software

Gabriel Nogueira Vieira Resende Vitor Costa Vianna
Joaquim Guilherme de Carvalho Vilela Silva

2026, v-1.0.0

Resumo

A compreensão das características que definem o sucesso e a manutenção de sistemas open-source é fundamental para a engenharia de software contemporânea. Este relatório apresenta um estudo empírico baseado na mineração dos 1.000 repositórios mais populares do GitHub, avaliados pelo número de estrelas. Utilizando consultas automatizadas via API GraphQL, foram extraídas métricas referentes à idade, volume de contribuições externas (pull requests), frequência de releases, tempo de atualização, linguagens primárias e taxa de resolução de issues. Além da análise descritiva padrão, este trabalho introduz testes de hipóteses estatísticas não-paramétricas (Spearman, Mann-Whitney e Kruskal-Wallis) para validar correlações entre maturidade do projeto e engajamento da comunidade. Paralelamente à análise de dados, o documento detalha a configuração do processo de desenvolvimento da equipe, fundamentado na metodologia Kanban através do GitHub Projects, incluindo políticas de limite de Work in Progress (WIP) e rastreabilidade contínua de tarefas.

Palavras-chaves: Mineração de Repositórios de Software. Métricas de *Software*. Código Aberto. Estatística Não-Paramétrica. Kanban.

Introdução

Ecossistemas de código aberto são pilares da infraestrutura moderna de software. Projetos hospedados em plataformas como o GitHub não apenas democratizam o acesso a ferramentas complexas, mas também servem como laboratórios em tempo real para a observação de práticas de engenharia, governança de código e colaboração distribuída. Entender o que torna um repositório "popular" e como essa popularidade se traduz em saúde e sustentabilidade do software é um desafio constante para pesquisadores e engenheiros de plataformas.

O presente trabalho, desenvolvido no contexto da disciplina de Laboratório de Experimentação de Software, tem o duplo objetivo de executar a mineração de dados em larga escala e estabelecer um fluxo de trabalho estruturado e ágil para a equipe.

Na primeira frente, investigamos sete Questões de Pesquisa (RQs) predefinidas, analisando os 1.000 repositórios mais estrelados do GitHub para extrair e discutir métricas de maturidade, engajamento comunitário e ritmo de manutenção. Para elevar o rigor da pesquisa, estendemos a análise descritiva com o uso de métodos de inferência estatística, buscando comprovar matematicamente as correlações observadas e evitar conclusões baseadas em viés de amostragem.

Na segunda frente, documentamos a governança do próprio projeto. Adotamos o GitHub Projects (v2) como ferramenta de gestão, implementando um quadro Kanban com fluxo de estados definido, limites rigorosos de trabalho em progresso (WIP) e rastreabilidade ponta a ponta entre commits e issues. Essa estrutura garante não apenas a entrega iterativa das sprints, mas simula um ambiente de desenvolvimento profissional. Este exemplo deve ser utilizado como complemento do manual da classe

1 Hipóteses Informais

Esta seção apresenta as respostas esperadas baseadas na intuição antes da coleta empírica de dados, cobrindo as características fundamentais dos sistemas *open-source* em análise:

- **RQ01 (Maturidade/Idade):** Projetos populares tendem a ser antigos. A construção de uma comunidade engajada e a consolidação de uma base de código estável exigem anos de desenvolvimento contínuo.
- **RQ02 (Contribuição externa):** Sistemas populares recebem um alto volume de *pull requests*. A alta visibilidade atrai um ecossistema global de desenvolvedores dispostos a corrigir *bugs* e propor *features*.
- **RQ03 (Frequência de *releases*):** Repositórios populares lançam *releases* com alta frequência, refletindo a adoção de práticas modernas de integração e entrega contínuas (CI/CD) e a necessidade de entregar valor constante aos usuários.
- **RQ04 (Frequência de atualização):** Projetos populares são atualizados constantemente. Um baixo tempo desde a última atualização (geralmente em dias ou horas) é um indicativo de um projeto vivo e com manutenção ativa.
- **RQ05 (Linguagens populares):** A vasta maioria dos sistemas populares é escrita em linguagens consolidadas na indústria e nos *rankings* globais (como Python, JavaScript, Java e C++), refletindo a demanda do mercado de *software*.
- **RQ06 (Taxa de *issues* fechadas):** Existe uma alta proporção de *issues* fechadas em relação ao total. Isso indica uma governança técnica sólida e uma equipe de mantenedores com capacidade operacional para lidar com a demanda da comunidade.
- **RQ07 (Cruzamento de Linguagem vs. Engajamento):** Projetos escritos nas linguagens mais populares do mercado recebem mais contribuições externas e *releases*, pois a barreira de entrada é menor devido à abundância de desenvolvedores fluentes nessas tecnologias.

2 Hipóteses Extras (Testes Estatísticos)

Para estender a análise além da estatística descritiva, formulamos hipóteses inferenciais para testar matematicamente as correlações observadas no *dataset*. Dada a natureza típica das métricas de *software*, que não assumem uma distribuição normal, foram selecionados testes não-paramétricos (BORGES; HORA; VALENTE, 2016).

2.1 Idade vs. Engajamento (RQ01 e RQ02)

Avaliamos se o tempo de vida do repositório impacta a atração de contribuições da comunidade, utilizando o **Teste de Correlação de Spearman**.

- H_0 : Não há correlação monotônica entre a idade do repositório e o número total de *pull requests* aceitos.
- H_1 : Existe uma correlação monotônica positiva entre a idade do repositório e o número total de *pull requests* aceitos.

2.2 Maturidade vs. Resolução de Issues (RQ04 e RQ06)

Verificamos se repositórios mais maduros apresentam uma proporção diferente de resolução de problemas em comparação com repositórios mais jovens, utilizando o **Teste U de Mann-Whitney**. A maturidade foi definida com base na mediana da idade dos repositórios (em dias): projetos com idade acima da mediana foram classificados como *Maduros*, e os demais como *Jovens*.

- H_0 : A mediana da razão de *issues* fechadas é igual para repositórios maduros e repositórios jovens.
- H_1 : A mediana da razão de *issues* fechadas é diferente entre os dois grupos independentes.

2.3 Ecossistema de Linguagens (RQ07)

Analizamos se a linguagem primária afeta o engajamento da comunidade de forma significativa, isolando as três principais linguagens da amostra por meio do **Teste de Kruskal-Wallis**.

- H_0 : As medianas do total de *pull requests* aceitos são iguais entre as três linguagens primárias mais populares da amostra.
- H_1 : Pelo menos uma das linguagens possui uma mediana de *pull requests* estatisticamente diferente das demais.

3 Metodologia de Coleta

Para a realização deste estudo empírico, a coleta de dados foi automatizada por meio de um *script* desenvolvido pela equipe, consumindo diretamente a API GraphQL do GitHub. A consulta foi parametrizada para extrair os 1.000 repositórios com o maior número de estrelas (*stars*), garantindo o foco na "elite" do ecossistema *open-source*.

Para contornar os limites de paginação da API, o *script* implementou requisições iterativas utilizando o sistema de cursores do GraphQL, consolidando os dados brutos em um arquivo `.csv`.

Conforme o planejamento das *Sprints*, cada membro da equipe validou individualmente o *dataset* referente às suas Questões de Pesquisa (RQs). Durante esta etapa de sanitização, analisou-se a distribuição dos dados e a presença de *outliers*. Optamos por **manter todos os *outliers*** identificados pelo critério do intervalo interquartil (IQR), uma vez que valores extremos em métricas de repositórios populares (ex: projetos com dezenas de milhares de *pull requests*) refletem fenômenos legítimos do ecossistema e não erros de coleta. A escolha por métodos estatísticos não-paramétricos, baseados na mediana e em ordenações, torna a análise robusta a esses valores extremos sem a necessidade de descartá-los. Repositórios com valores ausentes em métricas críticas (como ausência de *issues* rastreáveis) foram tratados para evitar divisões por zero, garantindo a integridade dos cálculos das medianas e dos testes não-paramétricos subsequentes.

4 Resultados

A análise descritiva dos 1.000 repositórios coletados retornou os seguintes resultados numéricos para cada métrica estabelecida. Devido à natureza assimétrica dos dados de *software*, adotamos a mediana como medida de tendência central.

- **RQ01 (Maturidade/Idade):** A mediana de idade dos repositórios analisados é de **2.820 dias** (aproximadamente **7,73 anos**).
- **RQ02 (Contribuição externa):** A mediana do total de *pull requests* aceitos é de **765,5**.
- **RQ03 (Frequência de *releases*):** Os repositórios lançam, em mediana, **40 *releases***.
- **RQ04 (Frequência de atualização):** O tempo mediano desde a última atualização até a data da coleta foi de **0 dias**, indicando que a esmagadora maioria dos repositórios populares foi atualizada no mesmo dia da coleta.
- **RQ05 (Linguagens populares):** As linguagens primárias mais frequentes na amostra foram: **Python** (24,97%), **TypeScript** (19,06%) e **JavaScript** (12,05%).
- **RQ06 (Taxa de *issues* fechadas):** A mediana da razão entre *issues* fechadas e o total de *issues* é de **87,5%**.
- **RQ07 (Cruzamento de Linguagem vs. Engajamento):** Filtrando os sistemas pelas linguagens mais populares identificadas, observamos as seguintes medianas para *pull requests*, *releases* e dias desde a última atualização, respectivamente: **Python** (559,5 PRs; 20,5 *releases*; 0 dias), **TypeScript** (1.990,5 PRs; 134 *releases*; 0 dias) e **JavaScript** (630,5 PRs; 39 *releases*; 0 dias).

5 Discussão: Hipóteses vs. Resultados

Nesta seção, confrontamos as hipóteses informais preestabelecidas com os resultados descritivos, e aprofundamos a análise com os testes estatísticos inferenciais adotados.

5.1 Análise Descritiva (RQs 1 a 7)

- **RQ01 e RQ02:** A hipótese de que projetos populares são antigos e engajados foi **confirmada**, dado que a mediana de idade é de **7,73 anos** e a mediana de *pull requests* aceitos é de **765,5**, valores compatíveis com projetos consolidados e com comunidades ativas.
- **RQ03 e RQ04:** A expectativa de manutenção constante foi **confirmada**, observando-se um intervalo mediano de atualização de apenas **0 dias** (i.e., a maioria absoluta dos repositórios foi atualizada no mesmo dia da coleta). Além disso, a mediana de 40 *releases* corrobora a adoção de práticas contínuas de entrega.
- **RQ05, RQ06 e RQ07:** A suposição sobre o domínio de linguagens de mercado foi **confirmada**, com liderança de **Python** (24,97%), seguido por **TypeScript** (19,06%) e **JavaScript** (12,05%). A governança se mostrou **eficiente**, com **87,5%** de taxa mediana de resolução de *issues*. Ainda que a hipótese sobre linguagens de mercado tenha sido confirmada, o cruzamento da RQ07 revelou que *TypeScript* apresenta a maior mediana de PRs aceitos (1.990,5), sugerindo que o domínio quantitativo não se traduz linearmente em engajamento por projeto.

5.2 Análise Inferencial

Para validar matematicamente as dinâmicas de engenharia de *software* sugeridas, aplicamos estatística não-paramétrica (BORGES; HORA; VALENTE, 2016):

- **Idade vs. Engajamento (Spearman):** O teste de correlação de Spearman revelou $\rho = 0,226$ com valor-p $\approx 4,31 \times 10^{-13}$. Portanto, **rejeitamos** a hipótese nula, indicando que **existe** correlação monotônica positiva e estatisticamente significativa entre maturidade e volume de *pull requests*. A magnitude do coeficiente ($\rho \approx 0,23$) sugere, contudo, uma associação de intensidade fraca a moderada: a idade contribui para explicar o engajamento, mas não é o fator dominante.
- **Maturidade vs. Resolução de Issues (Mann-Whitney U):** Ao comparar repositórios “Maduros” (idade > mediana) e “Jovens” (idade $\leq$ mediana), obtivemos $U = 139.072$ com valor-p $\approx 8,83 \times 10^{-9}$. Isso demonstra estatisticamente que a taxa de fechamento de *issues* **é diferente** entre os dois grupos. As medianas revelam a direção do efeito: repositórios maduros apresentam mediana de **0,9094**, enquanto os jovens apresentam **0,8330**, sugerindo que projetos mais antigos tendem a manter uma governança de *issues* mais eficiente.
- **Linguagens vs. Engajamento (Kruskal-Wallis H):** A comparação do volume de PRs entre as três linguagens mais frequentes (Python, TypeScript e JavaScript) retornou $H = 48,44$ com valor-p $\approx 3,03 \times 10^{-11}$, evidenciando que a linguagem primária **influencia** o engajamento da comunidade de forma estatisticamente significativa. Consistentemente com a análise descritiva, TypeScript concentra a maior mediana de PRs (1.990,5) frente a Python (559,5) e JavaScript (630,5), indicando um ecossistema com padrões distintos de contribuição.

6 Configuração do Processo

Para gerenciar o fluxo de trabalho durante as *sprints*, a equipe utilizou o *GitHub Projects (v2)* seguindo os princípios da metodologia Kanban.

O quadro foi estruturado com as colunas **Backlog**, **To Do**, **Doing**, **Review** e **Done**. Para garantir o foco e mitigar o acúmulo de tarefas, implementamos uma política de *Work in Progress* (WIP) restrita à coluna **Doing**, permitindo um limite máximo de **3 cards** simultâneos (correspondente a uma tarefa por membro da equipe em andamento por vez).

A rastreabilidade foi mantida exigindo que todos os *commits* do repositório mencionassem a respectiva *Issue* de origem. Além disso, ao término de cada ciclo (S01, S02 e S03), o estado do quadro foi exportado para *snapshots* em formato *.csv*, refletindo o fluxo real da equipe.

Link do Repositório (Grupo): <<https://github.com/lab-medicao-experimentacao/lab01-repositorios-populares>>

Considerações finais

Este estudo empírico investigou as características que definem a "elite" do ecossistema *open-source*, analisando os 1.000 repositórios mais populares do GitHub sob a ótica de sete Questões de Pesquisa e três testes estatísticos não-paramétricos. Os resultados sustentam, em sua maioria, as intuições iniciais da equipe: repositórios populares tendem a ser maduros (mediana de 7,73 anos), altamente colaborativos (mediana de 765,5 *pull requests* aceitos), ativamente mantidos (mediana de 0 dias desde a última atualização) e eficientes na governança de *issues* (razão mediana de fechamento de 87,5%).

Do ponto de vista inferencial, os três testes aplicados rejeitaram suas respectivas hipóteses nulas com significância elevada ($p < 0,001$). A correlação de Spearman entre idade e volume de *pull requests* ($\rho = 0,226$) confirma uma associação positiva porém de intensidade fraca a moderada, indicando que o tempo de vida contribui para o engajamento, mas não o determina. O teste de Mann-Whitney U evidenciou que repositórios maduros mantêm uma razão de *issues* fechadas significativamente maior que os jovens (0,9094 vs. 0,8330), reforçando a hipótese de que a consolidação de processos de manutenção é um resultado do amadurecimento do projeto. Por fim, o teste de Kruskal-Wallis H revelou diferenças estatisticamente significativas no volume de contribuições entre Python, TypeScript e JavaScript, com destaque para TypeScript, que apresentou a maior mediana de PRs por projeto (1.990,5), sugerindo que dinâmicas específicas de cada ecossistema exercem influência mensurável sobre o padrão de contribuição da comunidade.

Entre as limitações do estudo, destacam-se: (i) o recorte transversal da coleta, que impede análises longitudinais sobre a evolução dos repositórios; (ii) a métrica *timeSinceLastUpdate* apresentou variação insuficiente na amostra (praticamente todos os repositórios foram atualizados no dia da coleta), o que exigiu adaptação da hipótese original de "Ativo vs. Dormente" para uma comparação entre "Maduros vs. Jovens"; e (iii) o critério de popularidade adotado (número de estrelas) captura visibilidade, mas não necessariamente qualidade técnica ou impacto real dos projetos.

Como trabalhos futuros, sugere-se a incorporação de séries temporais para acompanhar a trajetória dos repositórios ao longo dos anos, a inclusão de métricas de qualidade de

código (cobertura de testes, complexidade ciclomática) e a análise qualitativa dos *outliers* identificados, os quais frequentemente representam projetos disruptivos cujos padrões diferem da tendência central da amostra.

Paralelamente à contribuição empírica, o processo de desenvolvimento adotado, fundamentado em Kanban com limites explícitos de *Work in Progress* e rastreabilidade ponta a ponta entre *commits* e *issues*, mostrou-se adequado ao contexto de uma equipe pequena distribuindo tarefas ao longo de *sprints* curtas, e serve como referência para futuros trabalhos de mineração de repositórios conduzidos em contexto acadêmico.

Referências

BORGES, H.; HORA, H.; VALENTE, M. T. Understanding the factors that impact the popularity of GitHub repositories. In: IEEE. *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. [S.l.], 2016. p. 334–345. Citado 2 vezes nas páginas 3 e 5.